

Trabajo Práctico N° 9

Paradigmas No Convencionales - Paradigma Funcional

Lectura: Sebesta Cap. 15; Scott Cap. 11

Nota: Puede encontrar un IDE online para SML en el siguiente enlace (<https://sosml.org/editor>) y para Haskell en el siguiente enlace (<https://play.haskell.org/>).

Conceptos

1. Enumere ventajas y desventajas de la programación funcional frente al paradigma imperativo.
2. Defina *transparencia referencial*. Los lenguajes funcionales puros poseen transparencia referencial. Analice las ventajas de contar con esta propiedad.
3. ¿Cuál es la principal diferencia entre las variables en un lenguaje funcional y en uno del paradigma imperativo? ¿Qué ventajas y desventajas puede ver respecto a estas diferencias?
4. ¿Sería posible extender un lenguaje funcional puro con variables globales? En caso afirmativo, explique cuáles serían las consecuencias, y en caso negativo, justifique.
5. Analice por qué, en la programación funcional, los bucles son modelados sólo a través de la recursividad.
6. ¿Qué significa que las funciones sean *ciudadanos de primera clase*?
7. ¿Qué es una *forma funcional* o *función de alto orden*? Ejemplifique.
8. ¿Una función *curried* es una forma funcional/función de alto orden?
9. Si las funciones no son ciudadanos de primera clase, ¿es posible definir formas funcionales?
10. ¿Puede un lenguaje del paradigma funcional presentar aliasing? ¿Puede un programa en SML tener aliasing? ¿Y en Haskell?
11. Compare el mecanismo de *pattern matching* utilizado por los lenguajes funcionales frente al mecanismo de unificación utilizado por Prolog. Utilice al menos dos criterios de evaluación para realizar su comparación.

Casos de Estudio e Investigación

12. Muestre qué valores quedan ligados a `x` e `y`, después de las siguientes declaraciones en SML:

a.

```
val x = 8;  
val x = 8 : int  
val x = 10 and y = 2*x;
```

b.

```
val x = 8;  
val x = 8 : int  
val x = 10; val y = 2*x;
```

13. ¿Cuál es la diferencia entre el uso de `val` en SML y una asignación en Java?
14. ¿Python tiene semántica de valores para sus variables? Justifique si su respuesta fue afirmativa o muestre un contra-ejemplo en caso contrario.
15. Una sintaxis alternativa para la expresión condicional en SML puede ser definida como:

```
fun nuevoif (cond,e1,e2) = if cond then e1 else e2;
```


Explique por qué factorial trabaja mal si es implementado utilizando `nuevoif`.
16. ¿Qué problemas tendría SML si no utilizara algún tipo de evaluación perezosa? ¿En qué caso Python también debe recurrir a este tipo de evaluación? ¿Por qué?
17. ¿En qué consiste la inferencia de tipo de SML y Haskell? ¿Python tiene inferencia de tipos?
18. ¿Qué tiempo de ligadura tiene la ligadura entre una función y su tipo en SML? ¿Y en Python? Analice ventajas y desventajas de esta característica para cada lenguaje.

19. Considere los siguientes códigos Python (izquierda) y SML (derecha):

```
def f(x,y):                fun f(x,y) = x + y
    return x + y
```

Suponga que en ambos casos se invoca a la función `f` con 5 y `[1,2,3]`. ¿Qué efecto tiene en cada caso? ¿Cuál es la principal diferencia?

20. Dada la siguiente declaración de la función `concatenar`, muestre la expresión de tipos asociada a la función, e indique si es polimórfica y/o forma funcional.

```
fun concatenar(nil, L2) = L2
    concatenar(cabeza::cola, L2) = cabeza :: concatenar(cola,L2);
```

21. Defina en Haskell una función `duplicar` que, dada una lista $L = [x_1, x_2, \dots, x_n]$, genere una nueva lista $L' = [x_1, x_1, x_2, x_2, \dots, x_n, x_n]$. Por ejemplo:

```
duplicar [0,3,1,4]    →    [0,0,3,3,1,1,4,4]
```

Muestre el tipo de la función `duplicar`, indique si es polimórfica y si hace uso de alguna forma funcional.

22. ¿Cuál sería el principal problema que le generaría al sistema de tipos de SML remover la inferencia de tipos? ¿Cómo podría solucionarse tal problema? ¿Qué ventajas y desventajas traerían estos cambios? ¿Tiene Haskell el mismo problema?
23. ¿Cuál es el tipo de las siguientes funciones curried? ¿Son polimórficas? ¿Son formas funcionales? ¿Qué funciones resultan de la aplicación parcial (mostrada entre paréntesis) de las siguientes funciones curried?

- a. `fun suma i j : int = i+j; (suma 3)`
- b. `fun menor a b : real = if a <b then a else b; (menor 2.0)`
- c. `fun f1 x y z = if (x = y) then x*z else z-y; (f1 2)`

24. Compare la dureza del sistema de tipos respecto a las expresiones mixtas de Haskell con la de Pascal y la de Python.

25. Para cada una de las siguientes funciones, indique su expresión de tipo, si son polimórficas y si son formas funcionales. Luego, impleméntelas en Haskell.

- a. `five`, dado cualquier valor, devuelve 5.
- b. `apply`, toma una función y un valor, y devuelve el resultado de aplicar la función al valor.
- c. `id`, la función identidad.
- d. `first`, que toma un par ordenado, y devuelve su primera componente.
- e. `swap`, que toma un par y devuelve el par con sus componentes invertidas.
- f. `xor`, el operador de disyunción exclusiva.
- g. `max3`, toma tres números enteros y devuelve el máximo entre ellos.

26. Sin usar funciones de alto orden, defina en Haskell las siguientes funciones y en cada caso indique su tipo, si son polimórficas y si son formas funcionales:

- a. `suma`, suma todos los elementos de una lista de números.
- b. `cuadrados`, calcula la lista de los cuadrados de los elementos de una lista de números dada.
- c. `longitudes`, dada una lista de listas, devuelve la lista de sus respectivas longitudes.
- d. `ordenados`, dada una lista de pares de números, devuelve la lista con aquellos pares en los que la primera componente es menor que el triple de la segunda.
- e. `pares`, dada una lista de enteros, devuelve una lista con los elementos pares.
- f. `masDe`, dada una lista de listas y un número, devuelve la lista con aquellas listas cuya longitud es mayor al número dado.

27. Usando la función de alto orden `map`, defina las funciones `cuadrados` y `longitudes` del ejercicio anterior. En cada caso indique su tipo, si son polimórficas y si son formas funcionales.

28. Compare los mecanismos para definir nuevas funciones como aplicaciones parciales de Python y SML. Utilice al menos dos criterios de los estudiados para realizar la comparación.

29. Defina en Haskell de manera curried la forma funcional **filter**, la cual aplica un predicado a los elementos de una lista y retorna la lista formada por los elementos que satisfacen el predicado. Por ejemplo:
`filter par [1,3,6,10,15,20] → [6,10,20]`
30. Aplicando parcialmente la función de alto orden **filter**, defina las funciones **pares** y **ordenados** del ejercicio 26. Indique su tipo, si son polimórficas y si son formas funcionales.
31. Considere las siguientes expresiones de tipos de funciones en SML. Determine si las funciones son: (1) función curried; (2) forma funcional; (3) función polimórfica.
- `f1: 'a list ->'b list ->int list`
 - `f2: (int * int * bool) ->int`
 - `f3: 'a list * ('a * 'a ->'a list) * 'b list * ('b * 'b ->'b list) ->int`
 - `f4: int list ->(int ->int) ->int`
32. Explique de qué forma difiere el proceso de inferencia de tipos en Haskell en comparación con SML. Indique las principales razones detrás de esas diferencias. Ilustre con ejemplos a partir de las funciones desarrolladas en los ejercicios 25 y 26.
33. Una definición del tipo árbol binario en SML puede ser la siguiente:
- ```
datatype 'a arbolb = hoja of 'a | nodo of 'a arbolb * 'a * 'a arbolb;
```
- Y si queremos que los elementos en las hojas puedan ser de un tipo diferente al de los nodos internos, podemos utilizar la siguiente definición:
- ```
datatype ('a, 'b) arbolb = hoja of 'b  
| nodo of (((('a, 'b) arbolb) * 'a * (('a, 'b) arbolb)));
```
- Escriba la expresión de tipos para una función que, dado un árbol, genere una lista con todos los elementos almacenados en las hojas del árbol. ¿Sería posible generar una lista con todos los elementos y en las hojas y en los nodos internos de este árbol?
34. Dada la siguiente secuencia de declaraciones en SML:
- ```
fun por2 x = x*2
val funcion1 = map por2;
fun funcion2 lista = map por2 lista;
```
- ¿Cuáles son las diferencias y similitudes entre *funcion1* y *funcion2*?
35. ¿En qué maneras SML se aleja de los lineamientos del paradigma funcional? Considerando que Haskell es un lenguaje funcional puro, ¿cómo hace para mantenerse dentro del paradigma en relación a esas características?